

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	5
1.1 Описание входных данных.....	7
1.2 Описание выходных данных.....	7
2 МЕТОД РЕШЕНИЯ.....	8
3 ОПИСАНИЕ АЛГОРИТМОВ.....	10
3.1 Алгоритм конструктора класса Item.....	10
3.2 Алгоритм деструктора класса Item.....	10
3.3 Алгоритм метода incDurability класса Item.....	11
3.4 Алгоритм метода decDurability класса Item.....	11
3.5 Алгоритм метода getName класса Item.....	11
3.6 Алгоритм метода getDurability класса Item.....	12
3.7 Алгоритм функции main.....	12
3.8 Алгоритм конструктора класса Item.....	13
3.9 Алгоритм конструктора класса Item.....	14
3.10 Алгоритм функции incDurability.....	14
3.11 Алгоритм функции dmgItem.....	15
3.12 Алгоритм функции brokenItem.....	15
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	16
5 КОД ПРОГРАММЫ.....	22
5.1 Файл Item.cpp.....	22
5.2 Файл Item.h.....	23
5.3 Файл main.cpp.....	23
6 ТЕСТИРОВАНИЕ.....	25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	26

1 ПОСТАНОВКА ЗАДАЧИ

В компьютерной игре существует система предметов. Каждый предмет имеет имя и показатель прочности. В процессе работы программы предмет может быть отремонтирован, повреждён или создан как сломанный объект.

Одни операции должны изменять исходный предмет, другие — работать только с его копией. Также необходимо предусмотреть ситуацию, при которой сломанный предмет может отсутствовать.

Требуется разработать программную систему, содержащую один класс. Класс должен включать:

- одно закрытое свойство строкового типа — имя предмета;
- одно закрытое свойство целочисленного типа — прочность предмета;
- конструктор по умолчанию (устанавливает по-умолчанию имя предмета Unknown и прочность 0);
- конструктор с параметрами для установки имени и начальной прочности объекта;
- конструктор копии;
- деструктор;
- метод увеличения прочности на заданное значение;
- метод уменьшения прочности на заданное значение;
- метод получения имени предмета;
- метод получения прочности предмета.

Конструкторы и деструктор должны выводить сообщения в следующем формате:

- Item (Cnstr): <имя предмета>
- Item (ParamCnstr): <имя предмета>

- Item (CopyCnstr): <имя предмета>
- ~Item (Dcnstr): <имя предмета>

Требуется реализовать следующие функции:

- Функция, принимающая объект **по ссылке** и увеличивающая его прочность на 10.
- Функция, принимающая объект **по значению** и уменьшающая его прочность на 5. Внутри функции повреждения вывести строку: "Inside damageItem: <имя> durability = <прочность>";
- Функция создания сломанного предмета принимающая в качестве параметра целое значение - режим работы, возвращающая **указатель на объект**:
 - о если введён режим 0, функция должна вернуть нулевой указатель;
 - о в противном случае функция должна создать динамический объект со значениями:
 - имя: Broken
 - прочность: 0и вернуть указатель на него.

Алгоритм работы программы

1. Ввести имя предмета, начальную прочность и режим работы.
2. Создать объект с использованием конструктора с параметрами.
3. Передать объект в функцию ремонта.
4. Вывести:After repair: <имя> durability = <прочность>
5. Передать объект в функцию повреждения.
6. После возврата из функции вывести:After damage: <имя> durability = <прочность>
7. Вызвать функцию создания сломанного предмета.
 - 7.1. Если возвращен нулевой указатель, вывести:Broken pointer

is nullptr

7.2. Иначе вывести: Broken item: <имя сломанного предмета>

durability = <прочность сломанного предмета>

8. Освободить динамическую память.

9. Завершить работу программы.

1.1 Описание входных данных

Пример ввода:

Sword 50 1

1.2 Описание выходных данных

Пример вывода:

```
Item (ParamCnstr): Sword
After repair: Sword durability = 60
Item (CopyCnstr): Sword
Inside damageItem: Sword durability = 55
~Item (Dcnstr): Sword
After damage: Sword durability = 60
Item (ParamCnstr): Broken
Broken item: Broken durability = 0
~Item (Dcnstr): Broken
~Item (Dcnstr): Sword
```

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект `item` класса `Item` предназначен для создания объекта класса;
- объект `cin` класса `istream` предназначен для ввода значения с клавиатуры;
- объект `cout` класса `ostream` предназначен для вывода значения на экран;
- функция `incDurability` для увеличения значения прочности;
- функция `dmgItem` для уменьшения значения прочности;
- функция `brokenItem` для создания сломанного предмета;
- `new` - оператор выделения динамической памяти;
- `delete` - оператор освобождения динамически выделенной памяти;
- `if... else` - условный оператор.

Класс `Item`:

- свойства/поля:
 - поле название предмета:
 - наименование — `name`;
 - тип — `string`;
 - модификатор доступа — `private`;
 - поле прочность предмета:
 - наименование — `durability`;
 - тип — `int`;
 - модификатор доступа — `private`;
- функционал:
 - метод `Item` — конструктор по умолчанию;
 - метод `Item` — конструктор с параметром;

- o метод `Item` — констрвктор копии;
- o метод `~Item` — деструктор;
- o метод `incDurability` — метод для увеличения значения прочности объекта;
- o метод `decDurability` — метод для уменьшения значения прочности объекта;
- o метод `getName` — метод для получения названия предмета;
- o метод `getDurability` — метод для получения прочности предмета.

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм конструктора класса Item

Функционал: конструктор по умолчанию.

Параметры: нет.

Алгоритм конструктора представлен в таблице 1.

Таблица 1 – Алгоритм конструктора класса Item

№	Предикат	Действия	№ перехода
1		зприсваивает закрытому свойству name значение "Unknown"	2
2		присваивает закрытому свойству durability значение, равное 0	3
3		выводит на экран "Item (Cnstr): <название предмета>"	∅

3.2 Алгоритм деструктора класса Item

Функционал: деструктор.

Параметры: нет.

Алгоритм деструктора представлен в таблице 2.

Таблица 2 – Алгоритм деструктора класса Item

№	Предикат	Действия	№ перехода
1		вывод на экран "~Item (Dcnstr): <название предмета>"	∅

3.3 Алгоритм метода incDurability класса Item

Функционал: метод для увеличения значения закрытого свойства (durability) объекта.

Параметры: int x - значение, на которое увеличиться прочность.

Возвращаемое значение: void, ничего не возвращает.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода incDurability класса Item

№	Предикат	Действия	№ перехода
1		увеличение настоящего значения durability на x	Ø

3.4 Алгоритм метода decDurability класса Item

Функционал: метод для уменьшения значения закрытого свойства (durability) объекта.

Параметры: int x - значение, на которое уменьшится прочность.

Возвращаемое значение: void, ничего не возвращает.

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода decDurability класса Item

№	Предикат	Действия	№ перехода
1		уменьшение параметра durability на x	Ø

3.5 Алгоритм метода getName класса Item

Функционал: метод для получения значения закрытого свойства.

Параметры: нет.

Возвращаемое значение: string, возвращает закрытое свойство name.

Алгоритм метода представлен в таблице 5.

Таблица 5 – Алгоритм метода *getName* класса *Item*

№	Предикат	Действия	№ перехода
1		возвращение закрытого свойства name	Ø

3.6 Алгоритм метода *getDurability* класса *Item*

Функционал: метод для получения значения закрытого свойства.

Параметры: нет.

Возвращаемое значение: int, возвращает закрытое свойство durability.

Алгоритм метода представлен в таблице 6.

Таблица 6 – Алгоритм метода *getDurability* класса *Item*

№	Предикат	Действия	№ перехода
1		возвращение закрытого свойства durability	Ø

3.7 Алгоритм функции *main*

Функционал: основной алгоритм программы.

Параметры: нет.

Возвращаемое значение: целое-индикатор корректности завершения программы.

Алгоритм функции представлен в таблице 7.

Таблица 7 – Алгоритм функции *main*

№	Предикат	Действия	№ перехода
1		объявление переменной name типа string	2
2		объявление переменной durability типа int	3
3		объявление переменной mode типа int	4
4		Ввод с клавиатуры переменных name, durability,	5

№	Предикат	Действия	№ перехода
		mode	
5		инициализация объекта item класса Item с параметрами: name, durability	6
6		вызов функции incDurability с параметром item	7
7		вывод на экран "After repair: <название объекта item> durability = <прочность объекта item>"	8
8		вызов функции dmgItem с параметром item	9
9		вывод на экран "After damage: <название объекта item> durability = <прочность объекта item>"	10
10		создание динамического объекта t класса Item и присвоение ему результата работы функции brokenItem с параметром mode	11
11	t == nullptr	Вывести на экран "Broken pointer is nullptr"	12
		Вывести на экран "Broken item: <название объекта t> durability = <прочность объекта t>"	12
12		освобождение динамически выделенной памяти по адресу t	∅

3.8 Алгоритм конструктора класса Item

Функционал: конструктор с параметрами.

Параметры: string name - название предмета; int durability - прочность предмета.

Алгоритм конструктора представлен в таблице 8.

Таблица 8 – Алгоритм конструктора класса Item

№	Предикат	Действия	№ перехода
1		присвоение значения name закрытому свойству name	2
2		присвоение значения durability закрытому свойству durability	3

№	Предикат	Действия	№ перехода
3		вывод на экран "Item (ParamCnstr): <название предмета>"	Ø

3.9 Алгоритм конструктора класса Item

Функционал: конструктор копии.

Параметры: const Item &item - ссылка на объект класса Item.

Алгоритм конструктора представлен в таблице 9.

Таблица 9 – Алгоритм конструктора класса Item

№	Предикат	Действия	№ перехода
1		присваивание закрытому свойству name значения свойства name объекта item	2
2		присваивание закрытому свойству durability значения свойства durability объекта item	3
3		вывод на экран "Item (CopyCnstr): <название предмета>"	Ø

3.10 Алгоритм функции incDurability

Функционал: увеличение значения.

Параметры: Item &item - ссылка на объект класса Item.

Возвращаемое значение: void, ничего не возвращает.

Алгоритм функции представлен в таблице 10.

Таблица 10 – Алгоритм функции incDurability

№	Предикат	Действия	№ перехода
1		вызов метода incDurability с параметром 10	Ø

3.11 Алгоритм функции **dmgItem**

Функционал: уменьшение значения.

Параметры: Item item - объект класса Item.

Возвращаемое значение: void, ничего не возвращает.

Алгоритм функции представлен в таблице 11.

Таблица 11 – Алгоритм функции *dmgItem*

№	Предикат	Действия	№ перехода
1		вызов метода decDurability с параметром 5	2
2		вывод на экран "Inside damageItem: <название предмета> durability = ∅ <прочность предмета>"	

3.12 Алгоритм функции **brokenItem**

Функционал: возвращение указателя на объект.

Параметры: int mode - режим работы.

Возвращаемое значение: Item* - ссылка на объект класса Item.

Алгоритм функции представлен в таблице 12.

Таблица 12 – Алгоритм функции *brokenItem*

№	Предикат	Действия	№ перехода
1	mode == 0	возвращение нулевого указателя	∅
		создание динамического объекта item класса Item с параметрами: "Broken", 0	2
2		возвращение объекта item	∅

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-6.

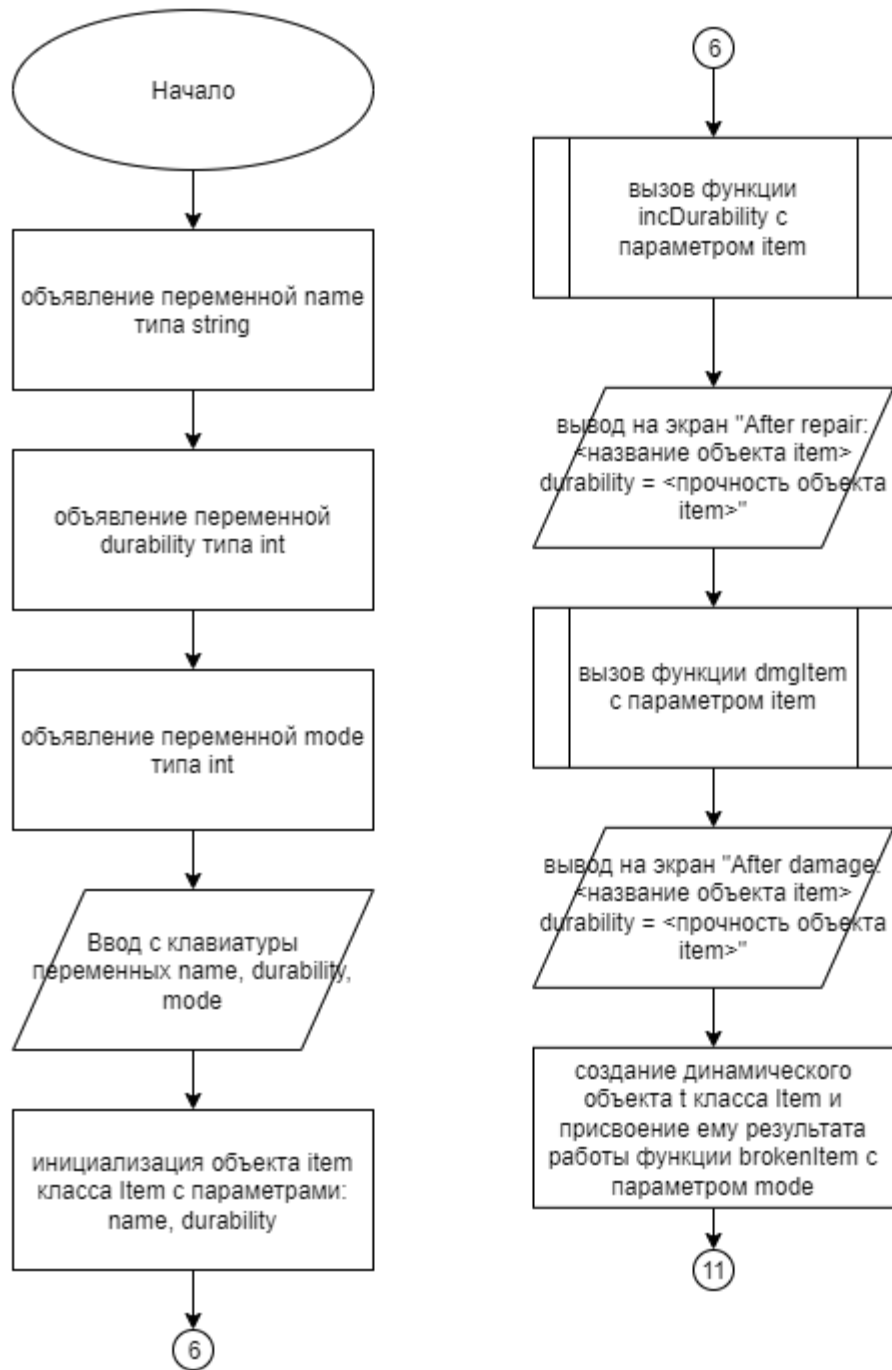


Рисунок 1 – Блок-схема алгоритма

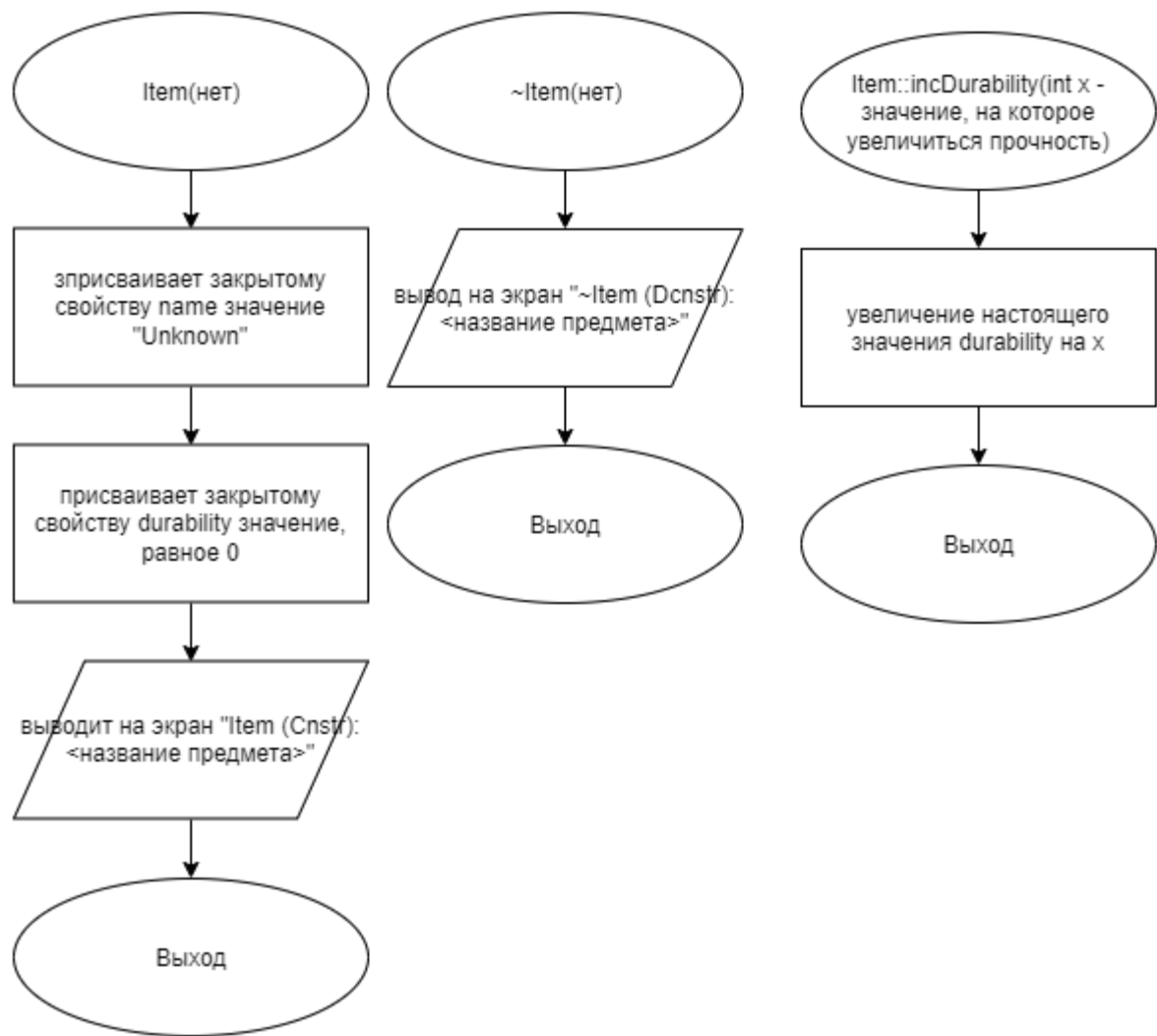


Рисунок 2 – Блок-схема алгоритма

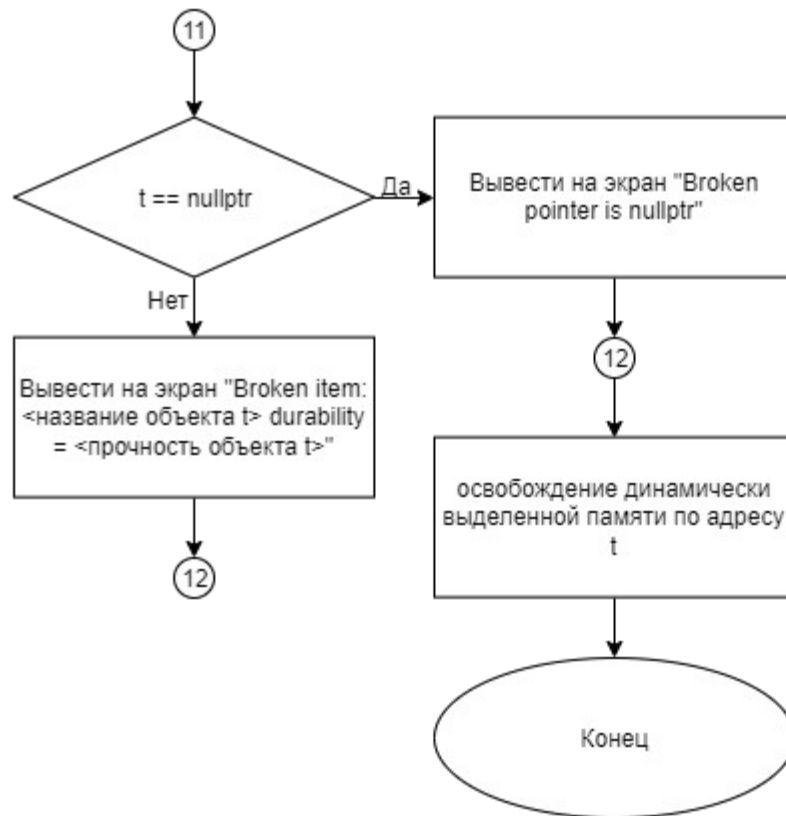


Рисунок 3 – Блок-схема алгоритма

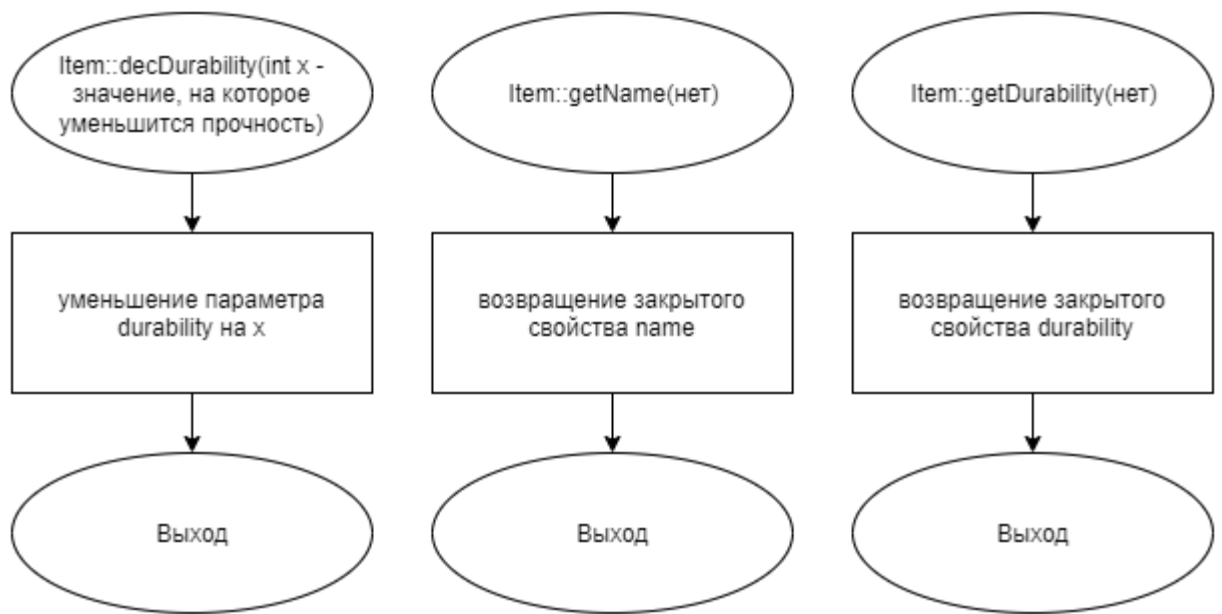


Рисунок 4 – Блок-схема алгоритма

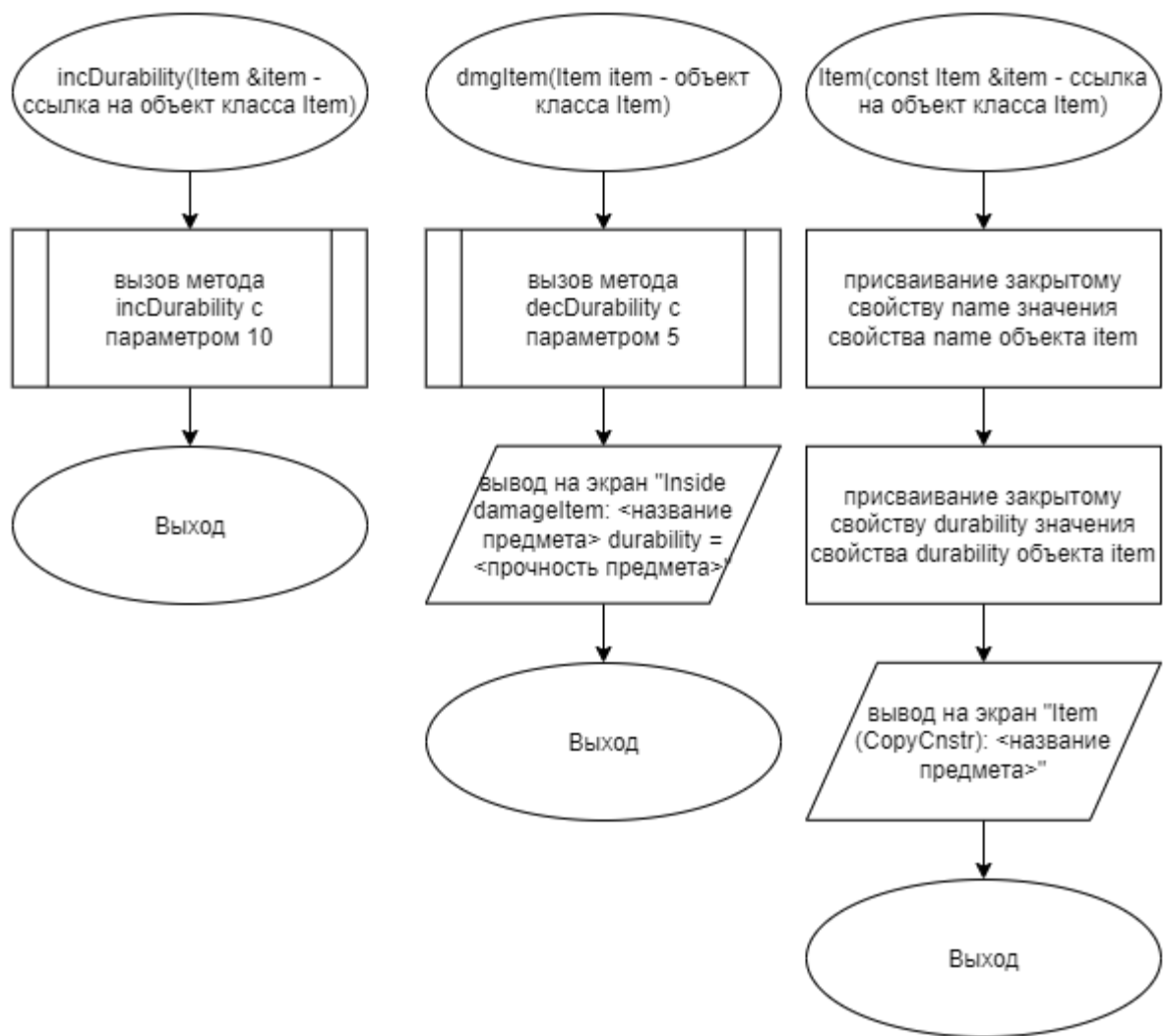


Рисунок 5 – Блок-схема алгоритма

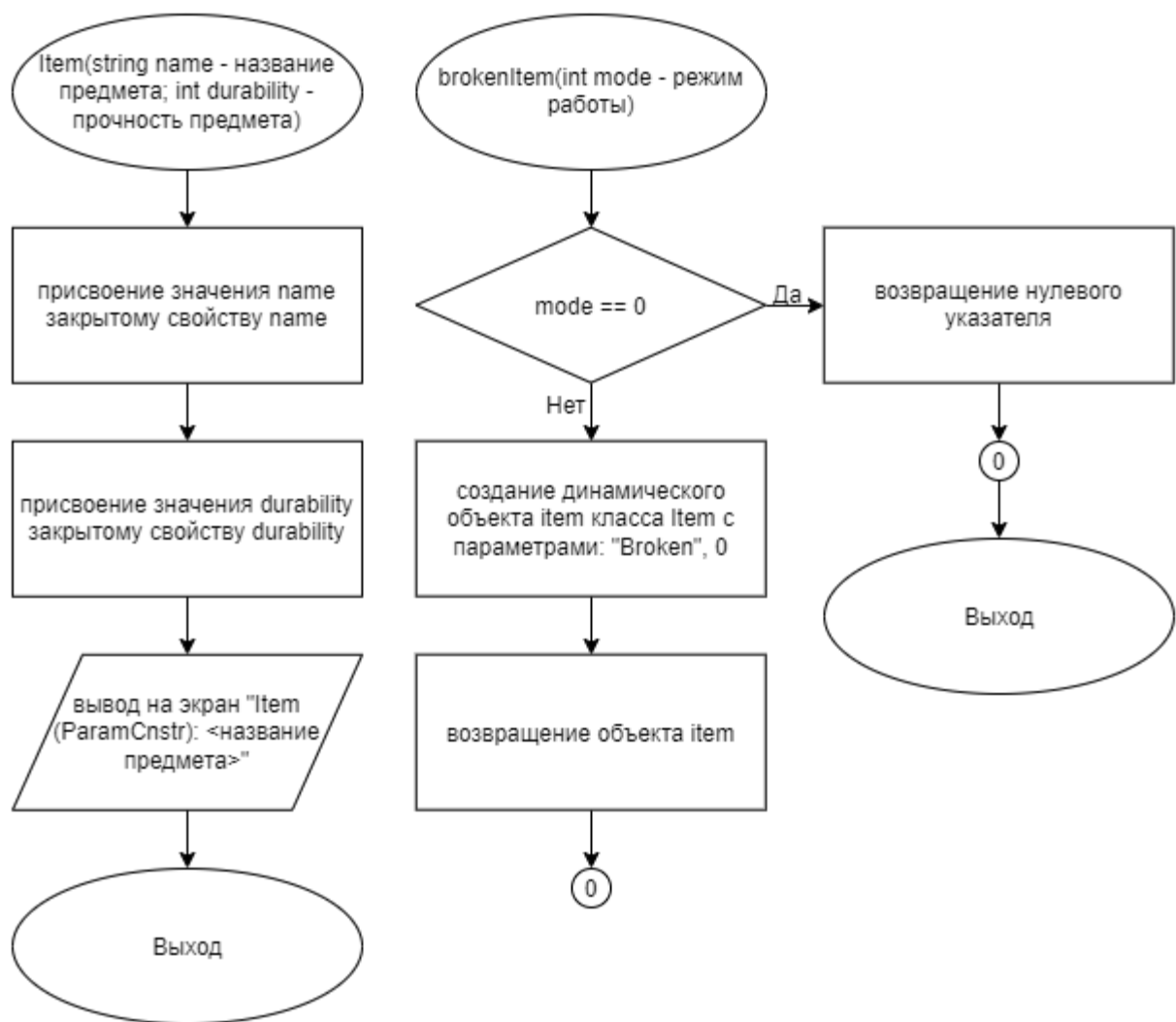


Рисунок 6 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл Item.cpp

Листинг 1 – Item.cpp

```
#include "Item.h"
#include <iostream>
using namespace std;

Item::Item()
{
    name = "Unknown";
    durability = 0;
    cout << "Item (Cnstr): " << name << endl;
}
Item::Item(string name, int durability)
{
    this->name = name;
    this->durability = durability;
    cout << "Item (ParamCnstr): " << name << endl;
}
Item::Item(const Item &item)
{
    name = item.name;
    durability = item.durability;
    cout << "Item (CopyCnstr): " << name << endl;
}
Item::~Item()
{
    cout << "~Item (Dcnstr): " << name << endl;
}
void Item::incDurability(int x)
{
    durability += x;
}
void Item::decDurability(int x)
{
    durability -= x;
}
string Item::getName()
{
    return name;
}
int Item::getDurability()
```

```
{  
    return durability;  
}
```

5.2 Файл Item.h

Листинг 2 – Item.h

```
#ifndef __ITEM__H  
#define __ITEM__H  
  
#include <iostream>  
using namespace std;  
class Item  
{  
private:  
    string name;  
    int durability;  
  
public:  
    Item();  
    Item(string name, int durability);  
    Item(const Item &item);  
    ~Item();  
public:  
    void incDurability(int x);  
    void decDurability(int x);  
    string getName();  
    int getDurability();  
};  
  
#endif
```

5.3 Файл main.cpp

Листинг 3 – main.cpp

```
#include "Item.h"  
#include <stdlib.h>  
#include <stdio.h>  
#include <iostream>  
using namespace std;  
  
void incDurability(Item &item)  
{
```

```

        item.incDurability(10);
    }
    void dmgItem(Item item)
    {
        item.decDurability(5);
        printf("Inside damageItem: %s durability = %d\n", item.getName().c_str(),
item.getDurability());
    }
    Item* brokenItem(int mode)
    {
        if (mode == 0)
        {
            return nullptr;
        }
        else{
            Item* item = new Item("Broken", 0);
            return item;
        }
    }
}
int main()
{
    string name;
    int durability;
    int mode;

    cin >> name >> durability >> mode;

    Item item(name, durability);

    incDurability(item);

    printf("After repair: %s durability = %d\n", item.getName().c_str(),
item.getDurability());

    dmgItem(item);

    printf("After damage: %s durability = %d\n", item.getName().c_str(),
item.getDurability());

    Item* t = brokenItem(mode);
    if (t == nullptr)
    {
        cout << "Broken pointer is nullptr\n";
    }
    else
    {
        printf("Broken item: %s durability = %d\n", t->getName().c_str(), t-
>getDurability());
    }
    delete t;
    return(0);
}

```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 13.

Таблица 13 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
Sword 40 1	Item (ParamCnstr): Sword After repair: Sword durability = 50 Item (CopyCnstr): Sword Inside damageItem: Sword durability = 45 ~Item (Dcnstr): Sword After damage: Sword durability = 50 Item (ParamCnstr): Broken Broken item: Broken durability = 0 ~Item (Dcnstr): Broken ~Item (Dcnstr): Sword	Item (ParamCnstr): Sword After repair: Sword durability = 50 Item (CopyCnstr): Sword Inside damageItem: Sword durability = 45 ~Item (Dcnstr): Sword After damage: Sword durability = 50 Item (ParamCnstr): Broken Broken item: Broken durability = 0 ~Item (Dcnstr): Broken ~Item (Dcnstr): Sword
Flail 33 0	Item (ParamCnstr): Flail After repair: Flail durability = 43 Item (CopyCnstr): Flail Inside damageItem: Flail durability = 38 ~Item (Dcnstr): Flail After damage: Flail durability = 43 Broken pointer is nullptr ~Item (Dcnstr): Flail	Item (ParamCnstr): Flail After repair: Flail durability = 43 Item (CopyCnstr): Flail Inside damageItem: Flail durability = 38 ~Item (Dcnstr): Flail After damage: Flail durability = 43 Broken pointer is nullptr ~Item (Dcnstr): Flail

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).